PROJECT ACTIVITY REPORT

Activity 4.2: Multimodal Signal Fusion

Project: Multimodal Attendance & Engagement Agent

1. Activity Overview & Objective

In this activity the signals from the Vision Module and the Audio Module are synchronized and combined into one unified data representation. Visual indicators (presence duration, movement patterns and inactivity periods) are aligned with audio indicators (noise levels and speech activity) using timestamp-based synchronization.

Core engineering objective. Fusion must let both modalities contribute proportionally to engagement understanding. By correlating visual attention cues with audio interaction levels, the system gets a far more accurate picture of classroom dynamics than either modality gives alone, and produces a structured, extensible input for higher-level reasoning and scoring.

2. Timestamp-Based Synchronization & Extensible Data Schema

Video frames and audio buffers are processed at different sampling rates, so raw events from the two streams cannot be compared directly. The fusion pipeline groups asynchronous events into discrete, timestamp-aligned windows and serializes each window into a standard dictionary, `session_data`, designed to take future modalities (such as digital interaction logs or wearable telemetry) without breaking existing consumers.

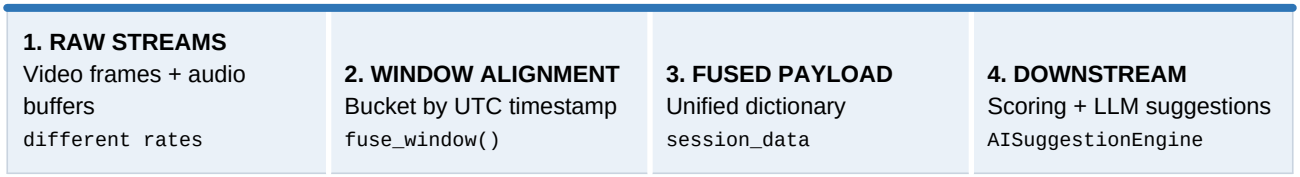

Figure 1: Timestamp-based multimodal signal fusion pipeline.

2.1 Synchronized Modality Indicators

Modality stream	Aligned indicators	Synchronization & aggregation logic	Proportional contribution
Vision Module	Presence duration, movement patterns, inactivity periods.	Frame-level bounding-box persistence and optical / pose motion deltas aggregated over sliding UTC timestamp windows.	Quantifies attendance stability, visual focus continuity, and motor restlessness or lethargy.
Audio Module	Ambient noise levels (dB), speech activity (VAD).	Audio energy envelopes and voice-activity segments resampled and bucketed to match the exact visual window timestamps.	Separates productive verbal participation and active lecturing from disruptive noise or passive silence.
Fused representation	Unified `session_data` payload.	Cross-modal correlation joining visual attention cues with concurrent acoustic interaction levels in an extensible schema.	Feeds deterministic scoring engines and LLM-based teaching assistants with holistic classroom context.

2.2 Input Sample Formats

Each stream emits small timestamped records (UTC epoch seconds). These field names are the ones used by the reference implementation below; adapt them to your Vision and Audio module outputs.

Stream	Record	Meaning
Vision	{ts, faces_visible, motion}	Faces currently tracked, and a 0-1 motion magnitude (optical flow or pose delta).
Audio	{ts, noise_db, speech}	Ambient level in dB and a voice-activity flag from the VAD.

2.3 Reference Implementation: SignalFusion

`SignalFusion` buckets samples into windows, computes per-modality indicators, and derives a cross-modal `context` label. A missing modality is reported honestly: the window is marked `single_modality` and the fused index renormalizes its weights instead of treating missing audio as silence.

File: backend/core/signal_fusion.py (reference implementation)

```
# File: backend/core/signal_fusion.py
def _clip(x, lo=0.0, hi=1.0):
    return max(lo, min(hi, x))


class SignalFusion:
    """Aligns vision and audio samples into timestamp-based windows and
    builds the unified session_data dictionary (UTC epoch seconds)."""

    def __init__(self, window_sec=30.0, motion_idle=0.05, noise_floor_db=30.0,
                 noise_ceiling_db=80.0, weights=None):
        self.window_sec = window_sec
        self.motion_idle = motion_idle            # motion below this = inactive
        self.noise_floor_db = noise_floor_db
        self.noise_ceiling_db = noise_ceiling_db
        self.weights = weights or {"vision": 0.5, "audio": 0.5}

    @staticmethod
    def _in_window(samples, start, end):
        return [s for s in samples if start <= s["ts"] < end]

    def fuse_window(self, vision, audio, start, end, expected_count):
        """vision: [{"ts", "faces_visible", "motion"}]
           audio:  [{"ts", "noise_db", "speech"}]"""
        v = self._in_window(vision, start, end)
        a = self._in_window(audio, start, end)
        data = {"window": {"start": start, "end": end}, "modalities": [],
                "vision": None, "audio": None}

        if v:
            data["modalities"].append("vision")
            data["vision"] = {
                "samples": len(v),
                "presence_ratio": _clip(sum(s["faces_visible"] for s in v)
                                        / (len(v) * max(expected_count, 1))),
                "mean_motion": sum(s["motion"] for s in v) / len(v),
                "inactivity_ratio": sum(s["motion"] < self.motion_idle for s in v) / len(v),
            }
        if a:
            data["modalities"].append("audio")
            mean_db = sum(s["noise_db"] for s in a) / len(a)
            data["audio"] = {
                "samples": len(a),
                "mean_noise_db": round(mean_db, 1),
```

```
                "noise_level": _clip((mean_db - self.noise_floor_db)
                                     / (self.noise_ceiling_db - self.noise_floor_db)),
                "speech_ratio": sum(bool(s["speech"]) for s in a) / len(a),
            }

        data["fusion_index"] = self._fusion_index(data)
        data["context"] = self._context(data)
        return data

    def _fusion_index(self, d):
        """Weighted blend; weights are renormalized if a modality is missing."""
        parts = {}
        if d["vision"]:
            parts["vision"] = d["vision"]["presence_ratio"] * (1 - d["vision"]["inactivity_ratio"])
        if d["audio"]:
            # productive talk OR calm quiet; loud non-speech noise scores low
            parts["audio"] = max(d["audio"]["speech_ratio"], 1 - d["audio"]["noise_level"])
        if not parts:
            return None
        total_w = sum(self.weights[k] for k in parts)
        return round(sum(self.weights[k] * parts[k] for k in parts) / total_w, 3)
    @staticmethod
    def _context(d):
        v, a = d["vision"], d["audio"]
        if not v or not a:
            return "single_modality"          # do not over-interpret
        if (v["presence_ratio"] >= 0.8 and v["inactivity_ratio"] <= 0.3
                and a["speech_ratio"] < 0.3 and a["noise_level"] < 0.4):
            return "quiet_concentration"
        if (v["presence_ratio"] < 0.6 or v["inactivity_ratio"] >= 0.6) \
                and a["speech_ratio"] < 0.2:
            return "disengagement_risk"
        if a["noise_level"] >= 0.7 and a["speech_ratio"] >= 0.5:
            return "active_interaction"
        return "mixed"

    def fuse_stream(self, vision, audio, start, end, expected_count):
        """Non-overlapping windows covering [start, end)."""
        out, t = [], start
        while t < end:
            out.append(self.fuse_window(vision, audio, t, min(t + self.window_sec, end),
                                        expected_count))
            t += self.window_sec
        return out
```

How the fusion behaves

- **Presence ratio:** average visible faces divided by the expected headcount in the window (attendance stability).
- **Inactivity ratio:** share of frames whose motion is below `motion_idle` (restlessness vs lethargy).
- **Speech ratio and noise level:** share of audio samples with speech, and the mean dB value scaled to 0-1.
- **Proportional index:** vision contributes presence x (1 - inactivity); audio contributes the larger of speech ratio and calm (1 - noise level). A quiet, focused room is therefore not penalized, while loud non-speech noise is.
- **Context label:** `quiet_concentration`, `active_interaction`, `disengagement_risk`, `mixed` or `single_modality`, which lets downstream logic tell quiet concentration from disengagement.

Example session_data (illustrative values)

```
{"window": {"start": 0, "end": 30},
 "modalities": ["vision", "audio"],
 "vision": {"samples": 30, "presence_ratio": 0.909,
            "mean_motion": 0.2, "inactivity_ratio": 0.0},
 "audio":  {"samples": 30, "mean_noise_db": 38.0,
            "noise_level": 0.16, "speech_ratio": 0.1},
 "fusion_index": 0.875,
 "context": "quiet_concentration"}
```

Behaviour check

The fusion code was run on 150 s of synthetic samples with 30 s windows. It produced `quiet_concentration` (index 0.875) for a calm, attentive window, `active_interaction` (1.0) for loud speech, `disengagement_risk` (0.45) for few visible faces with no motion or speech, `single_modality` when audio was missing, and `mixed` for loud non-speech noise. The thresholds and weights are illustrative and need calibration on real recordings; the code has not been run on real streams.

3. Downstream AI Suggestion Engine Implementation

Once the synchronized representation is built, it is passed to higher-level reasoning and scoring services. In `backend/core/ai_suggestions.py`, `AISuggestionEngine` consumes the fused payload and queries a language model to synthesize real-time teaching suggestions, with structured response parsing and deterministic exception handling. The source document names the model `llama3-8b-8192`; see Section 4 before using that ID.

3.1 Reference Implementation

The source listing shows only `generate_suggestions` and calls three helpers that are not shown. The version below keeps that method's structure and generation parameters (`temperature=0.7`, `max_tokens=200`) and adds the missing pieces.

File: backend/core/ai_suggestions.py (completed reference implementation)

```
# File: backend/core/ai_suggestions.py
import json
import os

SYSTEM_PROMPT = ("You are an expert AI teaching assistant. Use only the data "
                 "provided; do not invent facts. Reply with JSON only.")


class AISuggestionEngine:
    def __init__(self, client, model=None, temperature=0.7, max_tokens=200):
        self.client = client          # any OpenAI-compatible chat client
        self.model = model or os.getenv("LLM_MODEL", "")
        self.temperature = temperature
        self.max_tokens = max_tokens

    def _build_fusion_prompt(self, s: dict) -> str:
        v, a = s.get("vision") or {}, s.get("audio") or {}
        return (
            "Fused classroom data for one time window:\n"
            f"- Modalities available: {', '.join(s.get('modalities', [])) or 'none'}\n"
            f"- Presence ratio: {v.get('presence_ratio', 'n/a')}\n"
            f"- Inactivity ratio: {v.get('inactivity_ratio', 'n/a')}\n"
            f"- Mean motion: {v.get('mean_motion', 'n/a')}\n"
            f"- Mean noise (dB): {a.get('mean_noise_db', 'n/a')}\n"
            f"- Speech ratio: {a.get('speech_ratio', 'n/a')}\n"
            f"- Cross-modal context: {s.get('context', 'unknown')}\n"
            'Return JSON: {"suggestions": ["...", "..."]} with at most 3 short, '
            "actionable teaching suggestions."
        )

    def _parse_response(self, text: str):
        text = (text or "").strip()
        if text.startswith("```"):                        # strip code fences
            text = text.strip("`").removeprefix("json").strip()
        items = json.loads(text)["suggestions"]
        clean = [x.strip() for x in items if isinstance(x, str) and x.strip()]
        if not clean:
            raise ValueError("no suggestions in response")
        return {"suggestions": clean[:3], "source": "llm"}

    def _fallback_suggestions(self, s: dict):
        table = {
            "quiet_concentration": ["Students appear focused; allow quiet work to continue."],
            "disengagement_risk": ["Attention looks low; try a short question or activity.",
```

```
                                    "Check that all students can see and hear the lesson."],
            "active_interaction": ["Discussion is active; guide it toward the lesson goal."],
            "single_modality": ["Only one data source is available; confirm camera and microphone."],
        }
        out = table.get(s.get("context"), ["Class status is stable. Maintain current pace."])
        return {"suggestions": out, "source": "fallback"}

    def generate_suggestions(self, session_data: dict):
        prompt = self._build_fusion_prompt(session_data)
        try:
            completion = self.client.chat.completions.create(
                messages=[{"role": "system", "content": SYSTEM_PROMPT},
                          {"role": "user", "content": prompt}],
                model=self.model,
                temperature=self.temperature,
                max_tokens=self.max_tokens,
                timeout=10,
            )
            return self._parse_response(completion.choices[0].message.content)
        except Exception as e:
            print(f"AI Suggestion Error: {e}")
            return self._fallback_suggestions(session_data)
```

What was added to the source listing

- **Constructor and client injection:** any OpenAI-compatible client can be passed in, which also makes the engine easy to test with a mock.
- **_build_fusion_prompt:** turns `session_data` into a short prompt, writes `n/a` for missing modalities, and asks for JSON only.
- **_parse_response:** accepts plain or code-fenced JSON, keeps at most three non-empty strings, and raises an error on anything else so the fallback is used.
- **_fallback_suggestions:** deterministic advice chosen from the fused `context` label, so the dashboard still has useful text when the model fails.
- **Source flag and timeout:** results carry `source` = `llm` or `fallback`, and the call times out after 10 s.

Behaviour check

With a mock client, plain JSON and code-fenced JSON were parsed into suggestion lists. Non-JSON text, an empty list and a raised timeout each returned the deterministic fallback for the window's context. No live model was called.

4. Model Configuration Note

The source document specifies `llama3-8b-8192`, a Groq-hosted model ID. Groq's deprecation page lists a shutdown date of 30 August 2025 for it, with `llama-3.1-8b-instant` as the recommended replacement, so requests using the old ID are expected to fail. A later third-party report of Groq's deprecation page also lists `llama-3.1-8b-instant` as retired on 16 August 2026. Replacement models are themselves retired within months, so this code reads the model from the `LLM_MODEL` environment variable instead of hard-coding one.

Pick a currently supported model from your provider's model list, set it in `.env`, and re-check it before each release. If the model ID is wrong, the engine still returns fallback suggestions, so watch the `source` field to notice that the LLM is no longer being used.

5. Architectural Highlights

- **Proportional cross-modal correlation:** aligning presence duration and inactivity periods with speech activity prevents single-modality bias, so the engine can tell quiet concentration from disengagement.
- **Extensible data structure:** keeping fused indicators in one `session_data: dict` decouples upstream sensor synchronization from downstream reasoning. A new modality is added as another optional key (for example `"interaction_logs"`) that existing consumers can ignore.
- **Resilient inference pipeline:** `_parse_response()` and `_fallback_suggestions()` give usable feedback even under network or API failures. The LLM sits after the deterministic fusion step and cannot change the fused data.

6. Configuration Reference

Parameter	Default	Effect
window_sec	30 s	Window length. Shorter windows react faster but are noisier.
motion_idle	0.05	Motion below this counts as inactive.
noise_floor_db / noise_ceiling_db	30 / 80	Range used to scale noise to 0-1; calibrate to the room and microphone.
weights	vision 0.5, audio 0.5	Relative contribution to the fusion index; renormalized if a modality is missing.
temperature / max_tokens	0.7 / 200	From the source. A lower temperature usually gives more consistent JSON.
LLM_MODEL	(env var)	Model ID used by AISuggestionEngine.

7. Limitations & Future Improvements

Limitation	Impact	Improvement
Fixed windows with hard edges	Events near a boundary are split between windows.	Use overlapping (sliding) windows with a step smaller than the window.
Clock differences between devices	Misaligned streams give wrong correlations.	Use one clock source (NTP) and check for offset between audio and video.
Noise in dB depends on microphone and room	Thresholds do not transfer between classrooms.	Calibrate per room and normalize against a baseline.
Context rules are hand-set	Labels may be wrong in unusual sessions.	Calibrate on labelled recordings; extend the rule set.
Group-level data only	Cannot judge any one student.	Keep results at class level and avoid individual scoring.
Audio and video are personal data	Privacy and consent obligations.	Obtain consent, keep derived metrics instead of raw media where possible.

8. Validation Checklist

Tick each item after running it on your machine:

Check	How to verify	Status
Windows aligned	Samples fall into the correct 30 s window by timestamp	[ ]
Quiet concentration recognized	Attentive class, low noise: context is quiet_concentration	[ ]
Discussion recognized	Loud speech with faces visible: active_interaction	[ ]
Missing modality handled	Disable audio: single_modality and a valid index	[ ]
session_data schema stable	Adding a new key does not break the engine	[ ]
Fallback works	Wrong model ID or no network: source = fallback	[ ]
Live model call works	Valid LLM_MODEL set: source = llm and suggestions parsed	[ ]